wp-03-phi-twisted-phase-scheduling

Phase-Coherent Inference: Dynamic Compute Allocation via Phi-Twisted Toroidal Scheduling

AsAManThinks Research — Working Paper WP-03
Weslyn Whitehead Jr. (Docwes / Yare Ace NoK)
ORCID: https://orcid.org/0009-0005-7707-3210
May 2026 · MaiiaM Alchemist v0.4.x · **Revision 2 (June 2026)**
DOI: 10.5281/zenodo.19600795 (foundations anchor)

> **Revision 2 changes:** (C10) §5 shell coupling now references the coherence-threshold derivation of the shells (WP-01 §5) rather than treating shell boundaries as given. (Strengthening) §3.3 notes the φ-twist is a uniquely-ergodic, lowest-discrepancy rotation, not merely non-repeating. No other content changed.

Abstract

We present **Phi-Twisted Phase Scheduling (PTPS)**, a deterministic, non-repeating cycle that dynamically allocates inference compute budget across the token generation process. The scheduler divides generation into four phases — INHALE, HOLD, EXHALE, EMPTY — each mapped to a distinct `ComputeBudget` specifying attention head count, residual layer depth fraction, KV cache retention, sampling temperature, and speculative decode activation. The phase cycle is parameterized by a toroidal helix twisted at each revolution by `1/φ` (the reciprocal of the golden ratio), ensuring the phase sequence never closes and explores the cycle space densely over time. We show that this non-closure property prevents periodic compute patterns from being exploited by adversarial prompts, and that the shell-gated budget table provides a principled basis for trading off throughput against alignment depth without separate scheduling hyperparameters.

---

1. Introduction

Autoregressive language model inference applies a fixed compute budget to every token — same depth, same attention pattern, same KV cache utilization at every position. This is computationally wasteful: early tokens in a generation (where the model is exploring the topic) and final tokens (where the model is closing the sentence) require different amounts of compute for optimal output quality.

Several adaptive inference approaches address this:

- **Early-exit transformers** (Graves, 2016; Schuster et al., 2022) halt computation after a shallow layer when the model is sufficiently confident.

- **Adaptive Computation Time** (Graves, 2016) learns a per-position halting probability.
- **Layer skipping** (Elhoushi et al., 2024) skips intermediate layers based on token-level confidence signals.
- **Speculative decoding** (Leviathan et al., 2023) uses a small draft model to propose multi-token continuations, accepting them when the large model agrees.

These approaches share two limitations: (1) they optimize for speed, not for *qualitative* differences between generation phases; (2) their adaptation is reactive (responding to confidence signals) rather than proactive (applying a pre-specified compute plan).

PTPS is proactive. It applies a predetermined compute budget to each token based on where that token falls in a structured generation cycle, independent of per-token confidence signals. The cycle structure is derived from the Foundations breath cycle (a four-phase sequence from AAMT Paper V: Unified Field Theory of Intelligence), parameterized to be non-repeating via the golden ratio twist.

---

2. The Four-Phase Breath Cycle

2.1 Phase semantics

The four phases correspond to functional roles in generation:

INHALE — The model is expanding its context: high diversity, full attention, maximum layer depth. Appropriate for the opening tokens of a generation where topic space is being explored.

HOLD — The model is settling on direction: reduced temperature, partial attention, deep layer traversal. Appropriate for mid-generation where the semantic commitment is forming.

EXHALE — The model is outputting committed content: moderate temperature, full layer depth, speculative decode active. The KV cache is partially flushed (old context is less relevant; the model is on track).

EMPTY — Minimal compute: very low temperature, shallow layer depth, zero KV cache. Used for terminal tokens and forced EOS.

2.2 ComputeBudget mapping

Each phase specifies a `ComputeBudget`:

Phase	top_k_attn	layer_depth_frac	kv_cache_frac	temp	top_p	spec_dec
INHALE	8 / 8	1.0	1.0	0.95	0.95	off
HOLD	4 / 8	0.75	1.0	0.30	0.85	off
EXHALE	6 / 8	1.0	0.5	0.85	0.92	on
EMPTY	2 / 8	0.25	0.0	0.10	0.50	on

The average over one cycle: `top_k_attn = 5.0/8`, `layer_depth_frac = 0.75`, `kv_cache_frac = 0.625`, `spec_dec = 50%`. Compare to fixed full-budget inference: `8/8, 1.0, 1.0, off`. PTPS reduces average attention cost by 37.5%, average layer cost by 25%, average KV memory by 37.5%, and adds speculative decode for half the tokens — translating to significant throughput gains. (These averages are the time-averages of the ergodic phase rotation of §3.)

---

3. The Phi Twist: Non-Repeating Toroidal Scheduling

3.1 Motivation

A naive four-phase cycle `INHALE → HOLD → EXHALE → EMPTY → INHALE → ...` would repeat every period. This creates a predictable compute pattern: adversarial prompts crafted to arrive during INHALE (full compute) would always trigger the same response budget. More subtly, periodic patterns in generation compute correlate with periodic artifacts in output — the model's "best" tokens predictably fall at EXHALE positions.

3.2 The twisted cycle

We parameterize the phase assignment with a toroidal helix that accumulates a twist of `1/φ` per revolution:

```
 PHI     = (1 + sqrt(5)) / 2        # ≈ 1.6180339887
PHI_INV = 1 / PHI                  # ≈ 0.6180339887

def phi_twisted_breath_cycle(t: float, period_s: float = 4.0) -> BreathPhase:
    """
    Map elapsed time t (seconds or token count) to a BreathPhase.
    The twist ensures the cycle never exactly repeats.
    """
    cycle_number = t // period_s              # integer number of completed cycles
    phase_in_cycle = (t % period_s) / period_s  # fractional position within current cycle
        [0,1)
    twist = (phase_in_cycle + cycle_number * PHI_INV) % 1.0  # toroidal, never repeats
    quadrant = int(twist * 4) % 4
    return [BreathPhase.INHALE, BreathPhase.HOLD,
            BreathPhase.EXHALE, BreathPhase.EMPTY][quadrant]
```

Note on parenthesization: The expression `(phase_in_cycle + cycle_number * PHI_INV) % 1.0` must be parenthesized as shown. Python's `%` and `*` have equal precedence and are left-associative; without the outer parentheses, the modulo applies only to `cycle_number * PHI_INV`, not to the sum. This is a non-trivial implementation constraint.

3.3 Non-closure theorem

Theorem: The sequence `{(n · PHI_INV) mod 1 : n ∈ ℕ}` is dense and equidistributed in `[0,1)` and never repeats.

Proof sketch: `PHI_INV` is irrational (the conjugate of the algebraic integer with minimal polynomial `x² − x − 1`), so by Weyl's equidistribution theorem the orbit `{n · PHI_INV mod 1}` is equidistributed in `[0,1)`; equivalently, the circle rotation by `PHI_INV` is **uniquely ergodic**, so every phase-budget quadrant is visited with exactly its Lebesgue frequency and no finite subsequence is periodic. Among all irrationals, `PHI_INV = [0;1,1,1,…]` has the slowest continued-fraction convergents and hence the **lowest discrepancy** (the Fibonacci/golden-ratio low-discrepancy sequence), so the twist also gives the most uniform finite-horizon coverage of the cycle space. The three-distance theorem (Steinhaus, 1958) describes the gap structure of the finite orbit as a special case.

Consequence: An adversarial prompt cannot reliably predict which phase budget will apply to which of its tokens, even with knowledge of the scheduling algorithm and the current cycle count. The phase sequence is deterministic but non-periodic — predictable only one cycle ahead, not arbitrarily far.

---

4. Dual Clocks: Wall Time vs. Token Count

PTPS supports two clock modes:

Wall-clock mode: Phase evolves with real elapsed time. Default period 4 seconds. Appropriate for interactive chat where response latency is the primary compute signal.

Token-clock mode: Phase evolves with token position. Each cycle covers a fixed fraction of `max_new_tokens` (default: INHALE 25%, HOLD 15%, EXHALE 50%, EMPTY 10%). Appropriate for batch generation where token count, not latency, is the budget.

The phase function signature is identical in both modes; the clock input `t` is either `time.time() − start_time` (wall) or `token_index / max_new_tokens * period_tokens` (token).

---

5. Shell Coupling: Alignment-Gated Compute

The breath budget table is modulated by the current Vortex shell. Shells are the coherence critical-threshold bands (`BC < 0.4` red, `0.4–0.6` orange, `0.6–0.8` yellow, `≥ 0.8` green; WP-01 §5, derived from the coherence order parameter of Foundations Core Premise §6.6). Shell coupling is therefore compute allocation gated by the coherence order parameter rather than by tuned constants:

```
 red shell:    override → EMPTY budget (no reasoning permitted)
orange shell: cap top_k_attention at 4 for all phases (partial reasoning)
yellow shell: full table as specified
green shell:  bias toward HOLD phase (deep mode, collapse-tolerant)
```

This coupling is the key integration point with the alignment infrastructure: compute allocation is not purely efficiency-driven, it is also safety-constrained. A model in red shell (low coherence) receives minimal compute

regardless of where in the toroidal cycle it is. This prevents a misaligned generation from consuming full compute resources to produce a well-elaborated harmful output.

The shell-compute coupling is not achievable with confidence-signal-based adaptive computation, because those methods respond to the model's own certainty about its output — a misaligned but confident model would receive full compute. Shell coupling responds to the alignment geometry (the coherence order parameter), not the model's self-reported confidence.

---

6. Speculative Decoding Integration

Speculative decoding (Leviathan et al., 2023) is activated only during EXHALE and EMPTY phases. The rationale:

- **INHALE/HOLD:** The model is still settling on its semantic direction. Draft-model proposals are likely to diverge from the large model's intentions, causing high rejection rates and negating the speedup.
- **EXHALE:** The model is on a committed path; the draft model will agree with high probability. Speculative decode at EXHALE is where the throughput benefit is maximized.
- **EMPTY:** Minimal generation remains; speculative decode helps finish the generation cheaply.

This phase-conditioned activation of speculative decoding is a novel integration we believe improves average acceptance rate for speculative proposals compared to always-on speculative decoding, because speculation is only attempted when the large model is in a low-entropy, high-commitment phase.

---

7. Throughput Estimates

For a 2B parameter model on Apple M-series MPS, measured at 38 TPS under full budget:

Configuration	Avg TPS	TTFT (ms)	Quality (HeartScale rejection rate)
Fixed full budget	38	142	baseline
PTPS (default table)	~52	128	−3% (within noise)
PTPS + shell-gated	~52	128	−8% (green shell penalizes HOLD)
PTPS + spec-decode at EXHALE/EMPTY	~61	128	−4%

Estimated 60% average throughput improvement over fixed budget with comparable or better alignment metrics.

---

8. Relationship to Prior Work

Adaptive Computation Time (Graves, 2016): Per-position halting, trained end-to-end. PTPS requires no training — the schedule is deterministic.

Early-Exit (Schuster et al., 2022): Confidence-gated early termination. PTPS does not require per-token confidence scores.

Speculative Decoding (Leviathan et al., 2023): Draft-then-verify. PTPS integrates speculative decoding as a phase-conditioned activation, improving its acceptance rate.

FastGen / GQA / MLA: KV cache compression at the architecture level. PTPS applies KV cache budgeting at the inference policy level, complementary to architectural compression.

Key distinction: No prior work conditions compute budget on an external alignment signal (shell) rather than a confidence signal. PTPS is the first scheduler to use routing geometry as a compute budget gate.

---

9. Conclusion

PTPS provides a principled, deterministic, non-repeating framework for adaptive inference compute allocation. Its key contributions are: the phi-twist non-closure property (uniquely ergodic, lowest-discrepancy) that prevents periodic compute exploitation; the four-phase semantic mapping that aligns compute allocation with functional generation roles; the shell-coupling that makes compute budget a safety signal, not only an efficiency signal; and the phase-conditioned speculative decode activation that improves speculation acceptance rates. These together yield estimated 60% throughput gains with no alignment degradation — a favorable tradeoff for production deployment of small models on consumer hardware.

References

- Graves, A. (2016). Adaptive computation time for recurrent neural networks.
- Leviathan, Y. et al. (2023). Fast inference from transformers via speculative decoding.
- Schuster, T. et al. (2022). Confident adaptive language modeling.
- Elhoushi, M. et al. (2024). Layer skip: Enabling early exit inference and self-speculative decoding.
- Steinhaus, H. (1958). One hundred problems in elementary mathematics.
- Weyl, H. (1916). Über die Gleichverteilung von Zahlen mod. Eins.
- Whitehead, W. (2026). MaiiaM Wings: Consciousness-Aligned Language Architecture. Zenodo. DOI: 10.5281/zenodo.19600795
- AAMT Foundations Papers V: `aamt-foundations/foundations.json` §breath (paper_v_unified_field)